

[과제] 마이 헬스 로그 API 만들기

건강 기록을 관리하고 분석하는 나만의 API를 설계·구현·배포하는 개인 과제입니다.
아래 요구사항을 충족하는 API를 완성해 GitHub로 제출하세요.

과제 요약

- ✓ FastAPI로 건강 기록 관리 REST API 구현
- ✓ BMI 계산 · 혈압/혈당 분류 · 경고 · 통계 기능
- ✓ 데이터 파일 저장(재시작해도 유지)
- ✓ Docker 컨테이너로 실행 (배포는 가점)
- ✓ GitHub 저장소 + README로 제출

과제 형태

개인 프로젝트 (1인 1API)

진행 기간

4일 · 하루 4시간 (오후 2시~6시)

제출물

GitHub 저장소 URL (+ README)

참고 자료

FastAPI 수업자료 · 실습 워크북 · 배포 워크북

제출 정보

제출자 정보

먼저 본인 정보를 작성하세요

1 제출자 정보 (작성)

이름

소속 / 반

GitHub 저장소 URL

https://github.com/_____

배포 접속 URL (선택)

http://_____ : 8000/docs

제출일

20____ . ____ . ____

저장소는 Public으로

GitHub 저장소는 **Public**으로 만들고, 마지막 코드까지 `git push` 된 상태로 두세요.

과제 1

과제 개요 (미션)

무엇을 만들어야 하는지 확인합니다

미션: 매일의 건강 수치(몸무게·카·혈압·혈당 등)를 기록하면, 서버가 **BMI를 자동 계산하고 건강 상태를 분류**하며, 쌓인 기록으로 **통계**를 제공하는 API를 만드세요. 완성한 API는 **Docker로 실행**할 수 있어야 합니다.

1 이런 흐름으로 동작해야 합니다

사용자가 건강 수치를 POST로 전송

|

▼

서버가 저장 + BMI 계산 + 혈압/혈당 분류 + 경고 생성

|

▼

GET으로 기록·검색·통계를 JSON으로 응답

|

▼

Docker 컨테이너로 실행 (http://.../docs 로 테스트 가능)

학습용 프로젝트 안내

이 과제의 건강 분류 기준은 **학습을 위해 단순화**한 값으로, 실제 의학적 진단이 아닙니다. API 개발 학습이 목적입니다.

과제 2

필수 요구사항 — 기능 명세

아래 엔드포인트를 모두 구현해야 합니다

1 구현할 엔드포인트

메서드 · 경로	요구 동작
POST /records	건강 기록 추가. 저장 후 BMI·분류·경고를 계산해 응답
GET /records	전체 기록 조회 (개수 포함)
GET /records/{id}	기록 하나 조회. 없으면 404 반환
PUT /records/{id}	기록 수정
DELETE /records/{id}	기록 삭제
GET /search	날짜 범위(start, end)로 검색
GET /stats	평균 체중 등 통계 반환

2 공통 요구사항

- 요청 본문은 Pydantic 모델로 검증할 것
- 기록은 파일(JSON)에 저장해 서버를 재시작해도 유지될 것
- /docs (자동 문서)에서 모든 기능이 테스트 가능할 것
- 오류 없이 실행될 것 (실행 시 예외로 죽지 않을 것)

데이터 구조 & 분류 기준

이 규격에 맞춰 구현하세요

1 기록 데이터 필드 (필수)

필드	타입	설명
date	str	측정일 "2026-07-20"
weight	float	몸무게(kg)
height	float	키(cm)
systolic	int	수축기 혈압
diastolic	int	이완기 혈압
blood_sugar	int	공복 혈당(mg/dL)
steps / sleep_hours / memo	int / float / str	선택 (기본값 허용)

응답에는 위 필드에 더해 서버가 계산한 **bmi**, **bmi_category**, **bp_category**, **sugar_category**, **warnings**가 포함되어야 합니다.

2 구현해야 할 분류 기준

항목	구간	분류
BMI (kg ÷ m ²)	18.5 미만	저체중
	18.5 ~ 22.9	정상
	23 ~ 24.9	과체중
	25 이상	비만
혈압	수축기 <120 그리고 이완기 <80	정상
	120~139 또는 80~89	주의
	수축기 ≥140 또는 이완기 ≥90	고혈압
공복혈당	100 미만	정상
	100 ~ 125	공복혈당장애
	126 이상	당뇨 의심

3 경고(warnings) 규칙

- BMI가 **비만**이면 → 경고 메시지 추가
- 혈압이 **고혈압**이면 → 경고 메시지 추가
- 혈당이 **당뇨 의심**이면 → 경고 메시지 추가
- 해당 없으면 빈 목록 []

힌트

BMI = 몸무게(kg) ÷ (키(m) × 키(m)). 키가 cm이면 100으로 나눠 m로 바꾸세요. 구현 방법은 **실습 워크북**의 GET/POST·Pydantic·조건문 예제를 참고하세요.

단계별 수행 과제 (4일)

하루에 하나씩 완성하며 매일 커밋하세요

Day 1 — 프로젝트 세팅 & 기본 CRUD

✓ 완료 조건

- ☐ 가상환경 생성 & FastAPI 설치
- ☐ 기록 데이터 모델(Pydantic) 정의
- ☐ POST · GET(목록/단건) · DELETE 구현
- ☐ GitHub 저장소 생성 후 첫 커밋 push

Day 2 — 헬스케어 로직

✓ 완료 조건

- ☐ BMI 계산 & BMI/혈압/혈당 분류 구현
- ☐ 경고(warnings) 생성
- ☐ 조회 응답에 계산 결과 포함
- ☐ PUT(수정) 구현 & 커밋

Day 3 — 검색 · 통계 · 파일 저장

✓ 완료 조건

- ☐ GET /search (날짜 범위) 구현
- ☐ GET /stats (평균 등) 구현
- ☐ JSON 파일 저장으로 재시작해도 데이터 유지
- ☐ 커밋 (data.json은 .gitignore로 제외)

Day 4 — Docker & 제출 준비

✓ 완료 조건

- ☐ requirements.txt · Dockerfile · .dockerignore 작성
- ☐ docker build & docker run 성공
- ☐ README.md 작성
- ☐ (가점) 클라우드 배포 후 공인 IP 접속
- ☐ 최종 push 후 저장소 URL 제출

제출물 요구사항

저장소에 아래가 포함되어야 합니다

1 저장소 필수 파일

파일	설명
main.py	API 코드 (본인 작성)
requirements.txt	필요 패키지 목록
Dockerfile	이미지 빌드 설계도
.gitignore / .dockerignore	venv·data.json 등 제외
README.md	프로젝트 설명 문서

venv 폴더는 올리지 말 것

.gitignore로 `venv/` 를 제외하세요. venv는 용량이 크고 다른 PC에서 동작하지 않으므로 저장소에 올리지 않습니다.

2 README 필수 항목

- 프로젝트 소개 (무엇을 하는 API인지 2~3줄)
- 기능 목록 (엔드포인트 표)
- 실행 방법 (로컬 실행 & Docker 실행 명령)
- 기술 스택
- (배포했다면) 접속 URL

추가 도전 (선택)

필수 기능을 마쳤다면 나만의 기능으로 확장해 보세요

아래는 필수는 아니지만, 프로젝트를 더 돋보이게 만드는 확장 아이디어입니다. 하나 이상 구현하면 포트폴리오의 완성도가 올라갑니다.

- **목표 관리:** 목표 체중/혈압을 저장하고 달성을 반환
- **주간 리포트:** 최근 7일 평균과 지난주 대비 변화
- **걸음 수 등급:** 하루 걸음 수로 활동량 등급(부족/적정/우수)
- **수면 분석:** 평균 수면 시간과 권장 수면 비교
- **간단 화면:** HTML 한 페이지로 기록 입력/조회 (심화)
- **사용자 구분:** user 필드를 추가해 사용자별 기록 분리

기획서를 먼저 한 줄로

"누구의 어떤 문제를, 이 API가 어떻게 돕는가"를 README 맨 위에 한 줄로 적어두면 설득력이 커집니다.

최종 체크리스트 & 유의사항

제출 전에 반드시 확인하세요

✓ 제출 전 최종 체크

- ☐ 서버가 오류 없이 실행되고 /docs가 열린다
- ☐ 7개 엔드포인트가 모두 동작한다
- ☐ BMI·분류·경고·통계 결과가 올바르다
- ☐ 서버를 재시작해도 데이터가 유지된다
- ☐ docker build·run이 성공한다
- ☐ 저장소에 venv·data.json이 올라가지 않았다
- ☐ README가 작성돼 있다
- ☐ 최종 코드가 push됐고, 저장소가 Public이다

1 유의사항

- **본인 작성 원칙:** 코드는 스스로 작성하세요. 참고 자료를 봤다면 README에 밝혀도 좋습니다.
- **학습용 프로젝트:** 건강 기준은 단순화된 학습용 값이며 실제 진단이 아닙니다.
- **실행 가능성:** 저장소를 clone하면 바로 실행될 수 있어야 합니다. 제출 전 새 폴더에서 실행 테스트를 권장합니다.
- **막힐 때:** 수업 자료·실습 워크북·오류 해결표를 먼저 참고하고, 오류 메시지를 그대로 읽어보세요.

시작 템플릿 (선택)

막막하면 이 뼈대에서 시작하세요 (로직은 직접 채우기)

아래는 빈 뼈대입니다. **TODO** 부분을 요구사항과 기준표에 맞게 직접 완성하세요.

```
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel

app = FastAPI(title="마이 헬스 로그 API", version="1.0")

records = [] # TODO: Day 3에서 파일 저장으로 발전

class RecordIn(BaseModel):
    date: str
    weight: float
    height: float
    systolic: int
    diastolic: int
    blood_sugar: int
    steps: int = 0
    sleep_hours: float = 0.0
    memo: str = ""

# TODO: BMI 계산 / 분류 / 경고 함수를 작성 (기준표 참고)

@app.get("/")
def read_root():
    return {"message": "마이 헬스 로그 API"}

# TODO: POST /records - 기록 추가 (BMI/분류 자동 계산)
# TODO: GET /records - 전체 조회
# TODO: GET /records/{record_id} - 단건 조회 (없으면 404)
# TODO: PUT /records/{record_id} - 수정
# TODO: DELETE /records/{record_id} - 삭제
# TODO: GET /search - 날짜 범위 검색
# TODO: GET /stats - 통계
```

실행 방법

`uvicorn main:app --reload` 실행 후 `http://127.0.0.1:8000/docs` 에서 하나씩 확인하며 채워 나가세요.